

Section 14.7: FinTech Deep Dive

Part 7 of 8 in the System Design Interview Series

Executive Overview

FinTech systems demand correctness above all else. A single inconsistency means real money lost, regulatory exposure, or user trust destroyed. These systems combine distributed consensus for global coordination with ultra-low-latency optimisations for trading. Understanding double-entry bookkeeping, Saga patterns, idempotency, and fencing tokens at a deep level is what separates payment engineers from generalists.

Give extra weight to the correctness arguments in this section — numeric examples and proof-by-contradiction help in interviews.

1. Globally Distributed ACID-Compliant Payment Gateway

Requirements Analysis

Dimension	Requirement
Functional	Authorise, capture, settle payments across networks
Scale	100K+ TPS globally
Consistency	ACID across regions
Compliance	PCI-DSS, regional regulations (PSD2, NACHA)

Global Consistency — TrueTime Model (Google Spanner)

The fundamental challenge: in a globally distributed system, you cannot rely on wall-clock time for ordering (clocks drift). Google Spanner solves this with TrueTime.

TrueTime mechanics: - Each data centre has GPS receivers and atomic clocks - TrueTime returns an interval `[earliest, latest]` instead of a single timestamp - The uncertainty epsilon is typically

1–7ms - Before committing, a transaction waits until `now.earliest > all previous commit timestamps`

Proof of correct global ordering: - Transaction T1 commits at time t_1 - Transaction T2 starts after T1's commit is observable - T2's commit time t_2 is guaranteed to satisfy `$t_2.earliest > t_1.latest > t_1$` - Therefore `$t_2 > t_1$` globally — strict ordering preserved ✓

What this costs: An additional 1–7ms wait per cross-region transaction to let the uncertainty interval pass. For a payment that already takes 100–500ms end-to-end, this is acceptable.

Double-Entry Bookkeeping — The Correctness Foundation

Every financial system must use double-entry accounting. Every transaction has equal and opposite debits and credits. The sum of all ledger entries must always equal zero.

Transaction: Customer pays merchant \$100

Entries:

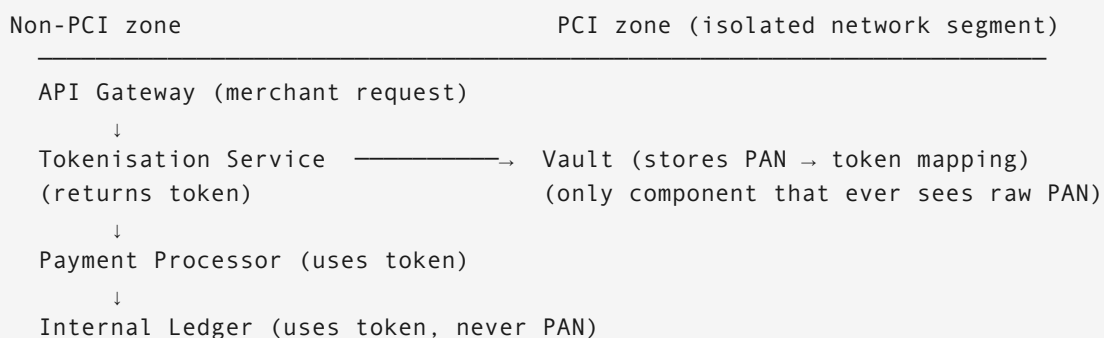
DEBIT	Customer.receivable	\$100.00
CREDIT	Merchant.payable	\$97.50
CREDIT	Interchange.fee	\$2.50

Sum: \$0.00 ← Must always balance

Violation proof by contradiction:

If entries don't balance, money was created or destroyed
→ Impossible in a real-money system → detect and reject all non-balancing transactions

PCI-DSS Scope Reduction



Benefit: Only the Vault is in PCI scope. All other services handle tokens, not real card numbers. PCI audit scope is dramatically reduced. If a non-Vault service is breached, no cardholder data is exposed.

Interview Problem 1 — Cross-Border Payment Reconciliation

“A payment is initiated in Singapore, processed via a bank in New York, and settled in London. All three systems have separate ledgers. How do you ensure the books balance globally, and what do you do when they don’t?”

Solution framework:

1. **Distributed saga:** Each leg is a separate saga step. Singapore leg: debit sender. New York leg: intermediary hold. London leg: credit recipient. Each step has a compensating transaction.
2. **Transaction reference propagation:** A globally unique transaction ID (UUID generated at origin) accompanies every message. Each system logs its entries under this ID. Reconciliation queries all three systems by this ID.
3. **Daily reconciliation batch:**

```
For each settlement date:
  Fetch Singapore ledger entries (all txns)
  Fetch New York entries for same txns (by txn ID)
  Fetch London entries
  For each txn: verify debits = credits across all three ledgers
  Flag mismatches > $0.01 for investigation
```

4. **Mismatch handling categories:**
 - **Timing difference:** New York hasn’t settled yet (normal, resolves next day)
 - **True discrepancy:** One leg failed silently → file a claim with the failing bank
 - **FX rounding:** Float arithmetic across currencies → accepted within \$0.01 tolerance
5. **Compensating transactions:** If London credit fails after Singapore debit succeeds, automatically initiate a refund saga back to the sender. All compensating txns logged with `COMPENSATING` flag for audit trail.

2. Ultra-Low Latency Electronic Trading Engine

Requirements Analysis

Dimension	Requirement
Functional	Match buy/sell orders; publish trade confirmations
Scale	10M+ orders/hour at market open

Dimension	Requirement
Latency	Sub-millisecond order-to-trade confirmation
Correctness	Price-time priority; no arbitrage opportunities

Limit Order Book (LOB) Data Structure

The central data structure of any exchange. Must support: - Insert order: $O(1)$ amortised - Cancel order: $O(1)$ with hash map lookup - Best bid/ask: $O(1)$ - Match at price level: $O(1)$ amortised (FIFO queue per price level)

```

Buy side (bids, descending):
  Price $100.50: [Order A: 500 shares] → [Order B: 200 shares] ← best bid
  Price $100.49: [Order C: 1000 shares]
  Price $100.48: [Order D: 300 shares]

Sell side (asks, ascending):
  Price $101.01: [Order E: 400 shares] ← best ask
  Price $101.02: [Order F: 600 shares]
  Price $101.03: [Order G: 800 shares]

Spread = $101.01 - $100.50 = $0.51

```

Price-time priority matching (worked example):

```

New sell order arrives: "Sell 700 shares at $100.50 or better" (market buy)

Match against buy side:
  Step 1: Order A at $100.50 has 500 shares → execute 500 shares at $100.50
           (Order A fully filled, removed from book)
  Step 2: Remaining 200 shares → Order B at $100.50 has 200 shares
           → execute 200 shares at $100.50 (Order B fully filled)
  Result: Full fill of 700 shares at $100.50

Trade reports:
  Trade 1: 500 shares @ $100.50 (OrderA vs NewOrder)
  Trade 2: 200 shares @ $100.50 (OrderB vs NewOrder)

```

Latency Optimisation Stack

Technique	Latency Reduction	Cost	Complexity
Kernel bypass (DPDK)	10–50µs → 1–2µs per packet	Medium	Medium

Technique	Latency Reduction	Cost	Complexity
FPGA order processing	Software 50µs → hardware 2–5µs	Very high	Very high
CPU pinning + NUMA	Eliminates cache coherence latency	Low	Low
Busy-wait polling	Eliminates interrupt latency	CPU waste	Low
Colocation at exchange	Eliminates network RTT	High (co-lo fees)	Low

Colocation physics: Speed of light in fibre = 200,000 km/s. 1km of fibre = 5µs RTT. Being 8km closer to the exchange saves 80µs — more than the entire processing time of some strategies.

Lock-free data structures: The LOB is accessed by multiple threads (order ingestion, matching, risk checks). Instead of mutex locks (which can pause threads for microseconds), use atomic compare-and-swap (CAS) operations. Lock-free skip lists allow concurrent reads and writes without blocking.

Interview Problem 2 — Exchange Matching Engine Design

“Design the core matching engine for a stock exchange handling 1M orders/second. How do you ensure price-time priority fairness while keeping latency below 1 millisecond?”

Solution framework:

1. **Single-threaded matching:** The matching engine for each instrument runs on a single dedicated CPU core (pinned). This eliminates all locking — only one thread modifies the LOB. Multiple instruments run on separate cores in parallel.
 2. **In-memory LOB:** The full LOB for each instrument fits in L3 cache (~50,000 price levels × 64 bytes = 3.2MB). Cache hits are nanoseconds.
 3. **Order ingestion:** Orders arrive via kernel-bypass network (DPDK). A lock-free ring buffer passes orders from the network thread to the matching thread without locking.
 4. **Risk pre-check:** Before the matching thread, a risk thread performs position/credit checks using read-only snapshots (updated every 100µs). Orders failing risk checks are rejected before reaching the LOB.
 5. **Determinism:** For fairness, if two orders arrive at the exact same microsecond, the one with the lower sequence number (assigned by the exchange’s order gateway) gets priority. Sequence numbers are assigned by a single atomic counter per gateway.
-

3. Design a Digital Wallet System

Requirements Analysis

Dimension	Requirement
Functional	Store value, send/receive, balance inquiry, transaction history
Scale	100M wallets, 10M transactions/day
Security	Cryptographic signing; fraud detection

Balance Representation — Event Sourcing vs. Current Balance

Approach	How It Works	Pros	Cons
Current balance column	<pre>UPDATE wallets SET balance = balance - amount</pre>	Simple, fast reads	Hard to audit; can drift
Event sourcing (ledger)	All transactions as immutable events; $\text{balance} = \Sigma(\text{events})$	Perfect audit trail; replayable	Slower balance reads
Hybrid	Ledger + materialised balance cache	Fast reads + full audit	More complex

Production standard: Hybrid. Immutable ledger for compliance and audit. Materialised `current_balance` updated atomically with ledger writes (in same transaction). Balance reads are $O(1)$. Full history always available by replaying ledger.

Double-Spend Prevention (Worked Example)

Account balance: \$50

Two concurrent payments: Payment A (\$40) and Payment B (\$30)

Without protection:

Thread 1 reads balance: \$50 → \$50 ≥ \$40 ✓ → deducts → balance = \$10

Thread 2 reads balance: \$50 → \$50 ≥ \$30 ✓ → deducts → balance = \$20

Result: Two deductions from \$50 = -\$20 overdraft (catastrophic)

With optimistic locking:

Thread 1: `UPDATE wallets SET balance = balance - 40, version = version + 1`
WHERE id = X AND balance ≥ 40 AND version = 7
→ SUCCESS, version now 8

```
Thread 2: UPDATE wallets SET balance = balance - 30, version = version + 1
        WHERE id = X AND balance >= 30 AND version = 7
        → FAIL (version is now 8, not 7) → retry
Thread 2 retries: reads balance = $10 → $10 < $30 → INSUFFICIENT_FUNDS
Result: No overdraft ✓
```

Proof by contradiction: If two concurrent debits both succeed with the balance check, they would each read the same initial balance and both pass the `balance >= amount` check. But with optimistic locking, the version check fails for the second committer — it cannot succeed unless the first transaction never committed. Therefore, exactly one of any two concurrent debits against the same account can succeed if it would cause overdraft.

Transaction Signing for Security

Payment initiated on mobile:

1. Client constructs payload: {to_wallet_id, amount, nonce, timestamp}
2. Nonce = random 32 bytes (prevents replay attacks)
3. Client signs payload with private key stored in device Secure Enclave
4. Server verifies signature against wallet's registered public key
5. Server checks nonce hasn't been used before (store nonces for 24h)

Attack prevention:

- Replay attack: nonce check rejects duplicate submissions
- MITM: server verifies signature → tampered payload rejected
- Key theft: private key never leaves Secure Enclave (hardware protected)

Interview Problem 3 — High-Throughput Payment with Full Audit Trail

“Design a payment system handling 10K TPS with exactly-once processing and a complete, immutable audit trail. How do you prevent double-spending and support instant balance queries?”

Solution framework:

1. **Idempotency layer:** Every payment request carries an idempotency key. Before processing, check a Redis cache (TTL 24h). If key exists, return stored result. This handles duplicate retries without double-processing.
2. **Sharded account locks:** With 10K TPS across many accounts, global locking would be a bottleneck. Shard accounts into 256 groups by `account_id % 256`. Each group has a dedicated lock manager. Concurrent payments to different accounts run fully in parallel.
3. **Atomic debit with ledger write:**

```

BEGIN;
-- Debit with optimistic lock
UPDATE accounts SET balance = balance - 100, version = version + 1
WHERE id = sender_id AND balance >= 100 AND version = current_version;
-- Check affected_rows; if 0, ROLLBACK

-- Credit receiver
UPDATE accounts SET balance = balance + 100, version = version + 1
WHERE id = receiver_id;

-- Write immutable audit entries
INSERT INTO ledger (txn_id, account_id, amount, type, created_at,
balance_after)
VALUES (txn_id, sender_id, -100, 'DEBIT', now(), new_sender_balance);

INSERT INTO ledger (txn_id, account_id, amount, type, created_at,
balance_after)
VALUES (txn_id, receiver_id, +100, 'CREDIT', now(), new_receiver_balance);
COMMIT;

```

4. **Instant balance query:** Reads from `accounts.balance` (materialised cache). Never computed from ledger sum at query time.
5. **Audit replay verification:** A nightly job independently sums all ledger entries per account and compares to `accounts.balance`. Any mismatch triggers an alert and manual review. This catches any bugs in the balance-update logic.

Section Summary: FinTech System Design Patterns

Requirement	Pattern	Correctness Guarantee
Exactly-once payment	Idempotency keys + ledger	Duplicate requests return stored result
Double-spend prevention	Optimistic locking + version check	Version mismatch fails second concurrent write
Global ordering	TrueTime commit wait	<code>t2.earliest > t1.latest</code> → strict order
Audit trail	Immutable double-entry ledger	Sum of all entries = 0 always
PCI compliance	Tokenisation + vault isolation	Non-vault components never see raw PAN
Stale lock owner	Fencing tokens at data store	

Requirement	Pattern	Correctness Guarantee
		<code>token < last_seen_token</code> → write rejected